

Initial File Triage

4/15/26

1:57 PM

Overall Triage Workflow(Summary)

Step-by-step order:

1. **Safe handling** — ZIP + infected password, rename extension.
2. **Hash the file** — sha256sum → search VirusTotal/MalwareBazaar.
3. **Run in sandbox** (ANY.RUN or Triage) — observe process tree, network, persistence.
4. **Static analysis** — file, strings, floss, hex editor - look for MZ (4D 5A), PE
5. **Network analysis** — suricata -r pcap → alerts.
6. **Check for packing** — missing strings, high entropy, section anomalies → use DIE.
7. **Unpack if needed** — upx -d or search memory dumps.
8. **Deobfuscate** — Base64 decode, .NET decompile, debug for config.
9. **Document IOCs** — hashes, domains, IPs, processes, family.

Final Mindset:

- Sandbox = fastest behavior insight.
- Dynamic analysis often bypasses obfuscation.
- Expect layers — iterate between static and dynamic.
- Goal: Extract IOCs fast for detection, hunting, and blocking.

1. Sandbox Basics & Malware Execution

When to use: First step when you have a suspicious file (doc, exe, script).

Why: Quickly see real behavior without risking your system. Combines **static** (file details) and **dynamic** (execution) analysis.

What You Upload & How It Works

- Upload the **malware/entry-point file** (e.g., malicious Word doc, .exe, .ps1).
- Sandbox runs it safely in a **virtual machine** (Windows 7/10, 32/64-bit).
- Records **everything**:
 - Processes & parent-child tree
 - Network traffic (HTTP, DNS, connections)
 - File drops & changes
 - Registry & memory activity

Key Output:

- File hashes (MD5, SHA256)
- Configuration extraction (e.g., Agent Tesla SMTP settings)
- Behavioral alerts (YARA matches)
- PCAP file (network traffic)
- Memory dumps & dropped files

Example Attack Chain (Agent Tesla):

1. Malicious Word doc opened → exploits Equation Editor (eqnedt32.exe).
2. Downloads payload via HTTP.
3. Executes via PowerShell.
4. Copies to Startup folder (persistence).
5. Checks geolocation (api.ipify.org).
6. Exfiltrates data via SMTP.

Why multiple VMs? Malware may only run on specific OS versions or require old software.

2. Safe Handling Practices

When to apply: Every time you acquire or move a malware sample. **Why:** Prevent accidental execution and spread.

Core Rules

- **ZIP archives:** Always password-protect with **infected**.
 - Prevents AV auto-deletion.
 - Clearly signals malicious content.
- **Rename extensions:** .exe → .bin or .exe.txt.
 - Stops OS auto-execution on double-click.
- **Lab setup:**
 - Dedicated host machine (never work/personal PC).
 - VMs (Windows 7/10, REMnux, Flare VM).
 - **Host-only** or air-gapped network.
 - Take VM snapshots (clean → pre-execution → during analysis).

Alternative for beginners: Use file **hash** only on VirusTotal or sandboxes instead of downloading the sample.

3. Core Static Analysis Tools (REMnux)

When to use: Immediately after receiving a sample (before running it).

Why: Verify file type, extract strings, and spot basic indicators without execution.

Tools & Commands

- **file** (Linux utility)
Bash
`file sample.bin`
Why: Reveals true file type (e.g., RTF instead of .doc, PE for Windows executable, .NET). Ignores fake extensions.
- **strings**
Bash
`strings sample.bin`
Why: Extracts printable ASCII strings (domains, IPs, commands). Limited to 1-byte characters.
- **floss** (better for obfuscated samples)
Bash
`floss sample.bin`
Why: Finds Unicode (UTF-16), wide-character, and obfuscated strings.
- **Hex editor** (e.g., wxHexEditor) **Why:** Views raw bytes. Look for magic bytes:
 - MZ (4D 5A) → Windows executable start.
 - PE\O\O (50 45 00 00) → PE header.

Key Insight: Few meaningful strings or mismatch with sandbox behavior → likely packed/obfuscated.

4. Gathering Basic Indicators (IOCs)

When to use: Early in every analysis. **Why:** Identify malware family, intent, and create detections/hunts.

Main Indicators

- **File:** Type, hashes, strings.
- **Network:** Domains, IPs, URLs, protocols.
- **Behavioral:** Processes, persistence, file drops.

Malware Stages

1. **Initial Access:** Macros, PowerShell, scripts (run once).
2. **Downloader:** Fetches next stage from internet.
3. **Dropper:** Contains payload inside (often .NET).
4. **Main Payload:** Info stealer, RAT, ransomware.

Platforms

- **VirusTotal:** Hash search, AV detections, behavior, tags.
- **MalwareBazaar** (abuse.ch): Cleaner family tags, free samples.

Goal: Quickly link to known families (e.g., Agent Tesla, RedLine, Amadey) for known capabilities.

5. Network-Based Indicators & Suricata

When to use: When you have a PCAP from sandbox or network capture.

Why: Reveals C2, downloads, exfiltration, and malware family.

Tool: Suricata

- Rule-based IDS.
- Works offline on PCAP files.

Commands

```
Bash
suricata-update          # Download Emerging Threats rules
suricata -r sample.pcap  # Analyze PCAP (offline mode)
```

Output

- eve.json: All events (alerts, HTTP, DNS, connections). Filter:
Bash
`grep "alert" eve.json`
`cat eve.json | jq .`

Alerts examples: PE download, C2 traffic, scripting language activity.

Limitation: No matching rule = no alert.

6. Basic Dynamic Analysis

When to use: After static triage to see real behavior. **Why:** Malware must de-obfuscate and act at runtime — often bypasses static hiding.

What to Observe

- **Processes:** Parent-child chains (e.g., winword.exe → PowerShell = suspicious).
- **File system:** Drops in AppData, Temp; mass file changes (ransomware).
- **Network:** C2, exfiltration.
- **Registry/Persistence:** Scheduled tasks (schtasks.exe).

Tools:

- Sandboxes (easiest).
- Custom VMs (more control).
- Debuggers (WinDbg, x64dbg) for advanced stepping.

Anti-Analysis: Malware detects sandbox/VM and stops or acts benign.

7. Advanced Sandbox Usage (ANY.RUN)

When to use: For interactive or real-time analysis. **Why:** See live process tree, network, and threats; correlate activity easily.

Key Features

- Real-time execution view.
- Choose environment (Win7–11, 32/64-bit).
- TLS interception → see HTTP content.
- Threat tags (e.g., Amadey, Loader).

Important Tabs

- **Process:** Hierarchical tree + full command lines (e.g., schtasks.exe for persistence).
- **Network:** HTTP (downloads), Connections (IP/port), DNS.
- **Threats:** Suricata IDS alerts.

Pro options: Custom VPN/TOR, FakeNet (simulate services without real internet).

8. File Hashing & Similarity

When to use: To identify or compare samples. **Why:** Hashes are unique fingerprints for searching and sharing IOCs.

Commands (REMnux)

```
Bash
md5sum file.exe          # Quick but weak
sha256sum file.exe       # Industry standard (MOST IMPORTANT)
sha512sum file.exe       # Stronger alternative
```

Fuzzy Hashing (ssdeep)

```
Bash
ssdeep file.exe > hash.txt
ssdeep -a -M hash.txt samples/ # Compare similarity %
```

Insight: Traditional hashes fail on packed/obfuscated variants. Fuzzy hashing finds related samples (percentage match).

9. Packing & Obfuscation Detection

When to use: When static analysis yields little (no strings, few imports).

Why: Packers hide the real payload — you analyze the wrapper instead.

Signs of Packing

- No meaningful strings (mismatch with sandbox).
- Very few imports.
- Abnormal sections (e.g., UPX0, UPX1; missing .text).
- High entropy (>7.0 — very random data).

Tools

- **Detect It Easy (DIE):** Detects known packers (shows signatures).
- **PE Studio:** Views sections, entropy, imports.

Unpacking (UPX example)

```
Bash
upx -d file.exe          # Unpack known UPX packer
```

Sandbox help: Search memory dumps (.dmp) for unpacked strings/domains.

Key Insight: Static ≠ dynamic behavior → almost always packed.

10. String & API Obfuscation + Cryptography

When to use: After unpacking or when strings are still hidden. **Why:**

Malware hides IOCs and behavior (domains, commands, APIs).

String Obfuscation

- **Base64:** Pattern A-Za-z0-9+/= (often ends with =). Extract & decode:
Bash
`strings --encoding=I file.exe | grep -E '[A-Za-z0-9+/=]{20,}'`
`base64 -d`
- Often layered with AES encryption.

API Obfuscation

- Dynamic resolution (call eax, dword_XXXX).
- Makes behavior hard to understand statically.

.NET Analysis

- Use **dnSpyEx** (decompiler + debugger on Flare VM).
- Go to entry point → find decode/decrypt functions.
- Set breakpoints → step over → read return values (e.g., C2 port, domains).

Configuration Extraction: Automate decoding to pull domains, ports, keys.